2.1(i) - Turn in this one

If (`bnsf-atsf > csx-kcs`), store `(bnsf-atsf)` into `cbq`.

```
la $t0, kcs
lw $s1, 0($t0)          # s1 = kcs

la $t0, erie            # s2 = erie
lw $s2, 0($t0)

la $t0, erie            # s3 = epsw
lw $s3, 0($t0)

sub $t1, $s1, $s2       # t1= bnsf-atsf

la $t0, csx
lw $s3, 0($t0)

la $t0, kcs
lw $s4, 0($t0)

sub $t2, $s3, $s4       # t2 = csx-kcs

slt $t3, $t2, $t1       # (t3 = csx-kcs < bnsf-atsf)
bne $t3, $zero, skipForward
la $t4, cbq
sw cbq, 0($t4)

skipForward:
____________
```

2.1(j) - Turn in this one

If (`kcs < erie` and `erie < epsw`), then store the value of `kcs` into `epsw`.

```
la $t0, kcs
lw $s1, 0($t0)          # s1 = kcs

la $t0, erie            # s2 = erie
lw $s2, 0($t0)

la $t0, erie            # s3 = epsw
lw $s3, 0($t0)

slt $t1, $s1, $s2       # t1 = ksc <erie
bne $t1, $zero, false   # if false, straight to post check

slt $t1, $s2, $s3       # t1 = erie < epsw
bne $t1, $zero, false   # if false, straight to post check

la $t1, epsw
lw $1, 0($t1)

false:
[Post Check Code]
```

2.2(e) - Turn in this one

Keep all of the even bits (0,2,4,6...). Do this in four instructions.

```
1010 1010 1010
0xAAAA
_______________

addi    $t1, $zero, 0xAAAA
andi    $s1, $s0, $t1
slli    $t1, $t1, 16
andi    $s1, $s1, $t1
```

2.3(d) - Turn in this one

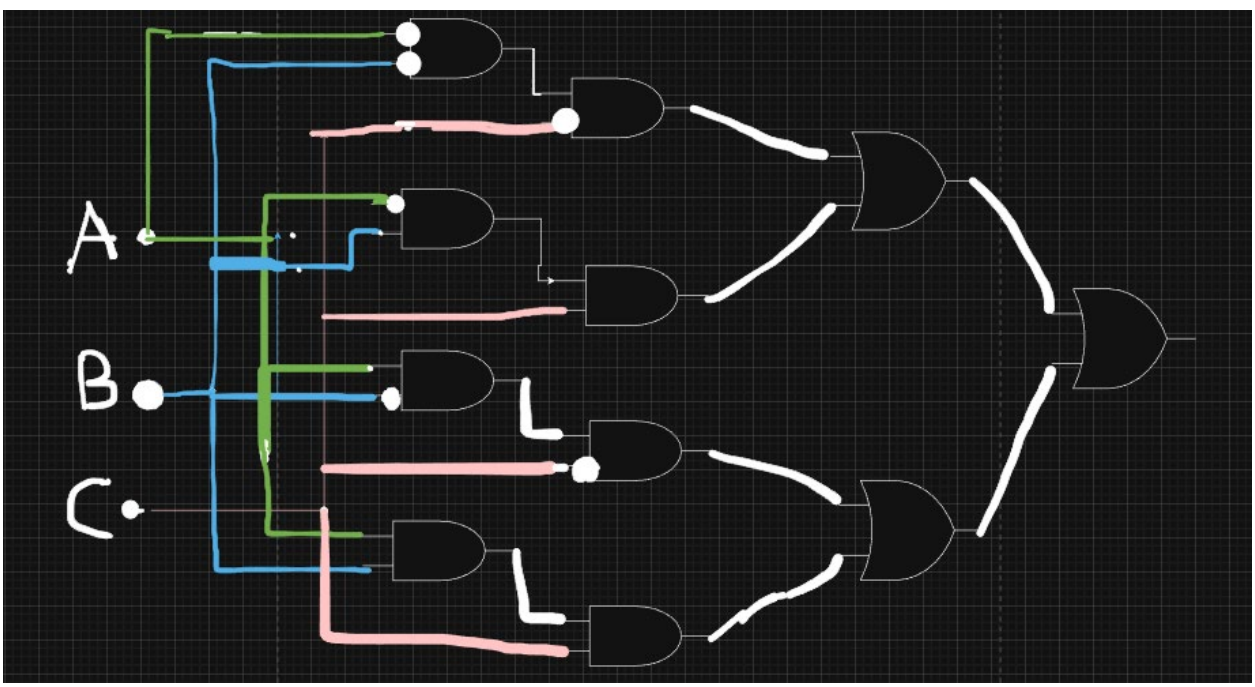
A	B	C	W	X	Y	Z
0	0	0	0	1	0	1
0	0	1	0	0	1	0
0	1	0	0	0	1	0
0	1	1	0	1	0	1
1	0	0	0	0	0	1
1	0	1	1	1	0	0
1	1	0	1	1	0	1
1	1	1	1	0	1	0

$$W = A\bar{B}C + AB\bar{C} + ABC$$

$$X = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}C + AB\bar{C}$$

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + ABC$$

$$Z = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + AB\bar{C}$$



2.4(d) - Turn in this one

C code:

```
int pow  = ... ;   // s0 - this is set by previous code

int prod = 1;      // allocate a register for this, of your choosing
for (int i=0; i<pow; i++)
{
    prod = prod*2;
}
```

```
addi $s1, $zero, 1      # s1 =  prod = 1
add  $t1, $zero, $zero  # t1 = 0


Loop:

slt $t2, $t1, $s0
bne $t2, $zero, outsideLoop

add $s1, $s1, $s1

addi $t1, $t1, 1

j Loop


outsideLoop:
[code]
```